ROTRTA: Ridiculous Optimizations Through Ridiculous Test Assertions — A Methodology for AI-Driven Performance Engineering, with Application to GPU Vector Similarity Search

IreGaddr
CatWhisper Project
iregaddr@gmail.com

***Abstract*—We introduce *Ridiculous Optimizations Through Ridiculous Test Assertions* (ROTRTA), a test-driven performance engineering methodology in which latency assertions are deliberately set beyond a system's current capability, anchored to the measured performance of a reference state-of-the-art system. Tests are written first and are contractually immutable: they may never be weakened to accommodate a lagging implementation. This asymmetry forces genuine optimization rather than satisficing, and provides AI-agent optimizers with a binary, reproducible oracle. We demonstrate ROTRTA on CatWhisper, a cross-vendor GPU vector similarity search library built on Vulkan. Starting from a manually optimized baseline that remained $2.4\times$ slower than FAISS-GPU across three benchmark configurations, a single ROTRTA session driven by an autonomous AI agent produced a stack of six coordinated optimizations — Structure-of-Arrays database layout, doubled warp-tile size, Vulkan timeline semaphores, userspace spin-poll synchronization, AVX-512 fp16 conversion, and zero-copy mapped I/O — that together reduced single-query latency by up to $18.9\times$ and caused CatWhisper to *beat FAISS-GPU on all three benchmark dimensions* simultaneously. We argue that ROTRTA is a broadly applicable discipline that makes latent performance budget visible, eliminates premature satisficing, and is uniquely well-suited to AI-agent-driven development because it reduces an open-ended optimization problem to a well-posed constraint satisfaction problem.**

***Index Terms*—test-driven development, performance engineering, vector similarity search, GPU computing, Vulkan, FAISS, AI agents, benchmark-driven optimization**

I. Introduction

High-performance systems development has long grappled with a fundamental tension: performance requirements are continuous and comparative, yet traditional test frameworks treat correctness as binary. A function either returns the right answer or it does not; but whether it returns it *fast enough* depends on hardware, workload, and what competitors have achieved. Standard benchmarking practices measure after the fact. Profiling-driven optimization follows an iterative, ad hoc loop — profile, identify hot spots, optimize, repeat. This loop is well understood for human developers. It is poorly matched to the working style of autonomous AI agents.

AI coding agents, in their current form, excel at implementing well-specified transformations and exploring large, locally constrained search spaces. They are less effective at formulating their own performance targets, recognizing satisficing behavior, or maintaining long-horizon optimization intent across many discrete decisions. Given a vague directive such as "make the search function faster," an agent may apply superficial improvements — reordering memory accesses, inlining a function — and declare success. Given a concrete, measurable, binary criterion — "the median single-query latency for 10,000 vectors at dimension 128 must be strictly less than $0.065\,\mathrm{ms}$" — the agent has an unambiguous oracle and cannot satisfice prematurely.

ROTRTA (**R**idiculous **O**ptimizations **T**hrough **R**idiculous **T**est **A**ssertions) formalizes this observation into a methodology:

1) **Measure the reference.** Establish the performance of the state-of-the-art system on the target hardware and workload.
2) **Write immutable assertions.** Encode performance targets as standard unit-test assertions, set at the reference or slightly below (25–50% reduction from the current implementation). The tests are written before the corresponding optimizations exist.
3) **Lock the tests.** Once written, test assertions are contractually immutable. They may never be weakened or deleted to make a failing implementation appear correct.
4) **Implement until the tests pass.** Any optimization technique that causes the assertions to pass is valid. Optimizations that address symptoms without curing root causes will reveal themselves when other assertions remain red.

In this paper we present ROTRTA as a discipline (Section III), describe CatWhisper — a Vulkan-based GPU vector search library and the system against which ROTRTA was applied — contrasting its design with FAISS (Section IV), trace the full optimization arc from baseline to ROTRTA-passing implementation (Section V), and analyze the experimental results (Section VI). We then discuss broader implications for AI-assisted software engineering (Section VII) and conclude (Section IX).

II. Background

A. Vector Similarity Search

Dense vector similarity search is a core primitive in modern information retrieval, recommendation systems, and retrieval-augmented generation (RAG) pipelines. Given a database $\mathcal{D} = \{v_1, \ldots, v_n\} \subset \mathbb{R}^d$ and a query $q \in \mathbb{R}^d$, the k-nearest neighbor (k-NN) problem finds the k elements of $\mathcal{D}$ minimizing a distance metric $\delta(q, v_i)$. For an exact flat index, this requires evaluating all n distances: $O(nd)$ arithmetic operations per query.

B. FAISS

Facebook AI Similarity Search (FAISS) [1] is the de facto reference library for vector similarity search. It provides both CPU and GPU backends, the latter accelerated by CUDA. FAISS implements multiple index families: IndexFlat (exact brute-force), IndexIVFFlat (inverted file), IndexIVFPQ (product-quantized IVF), and HNSW (hierarchical navigable small world graphs). GPU-accelerated IndexFlat achieves throughputs on the order of hundreds of millions of distance computations per second on modern NVIDIA hardware. As of FAISS 1.13.2 on an RTX 4080 Laptop GPU, single-query L2 search latency at $n = 10{,}000$, $d = 128$, $k = 10$ is $0.065\,\mathrm{ms}$; at $n = 100{,}000$, $d = 128$ it is $0.350\,\mathrm{ms}$; and at $n = 100{,}000$, $d = 256$ it is $0.601\,\mathrm{ms}$.

C. Vulkan Compute

Vulkan [7] is a low-overhead, cross-vendor GPU API standardized by the Khronos Group. Vulkan compute shaders, compiled to SPIR-V, execute on any Vulkan-capable GPU: NVIDIA, AMD, Intel, ARM Mali, and Apple Silicon (via MoltenVK). Vulkan 1.2 introduced timeline semaphores — monotonically-increasing 64-bit counters that eliminate the reset-before-reuse requirement of binary fences and enable userspace polling without kernel transitions on drivers that implement GPU-mapped semaphore surfaces.

D. AI Agents in Software Engineering

Large language model (LLM) agents have been applied to code generation, debugging, and refactoring [11]. Their utility for low-level performance optimization, however, has been less studied. Performance optimization requires hardware intuition, awareness of micro- architectural bottlenecks, and the ability to compose individually modest improvements into coordinated wins — capabilities that benefit substantially from a concrete, measurable success criterion.

III. The ROTRTA Methodology

A. Motivation: The Satisficing Problem

Optimization without a quantitative target converges to local minima. A developer (or agent) who achieves a 20% speedup has no mechanism to know whether 80% was achievable. Profiling can identify hot spots but does not indicate how much improvement is theoretically available. Conventional TDD enforces functional correctness but is silent on performance: a function that takes one second is as "correct" as one that takes one microsecond.

The ROTRTA insight is to close this loop by encoding performance targets as first-class, immutable test assertions, set to match or exceed a reference implementation. The choice of a reference that is marginally better than current performance (25–50% reduction) ensures the target is achievable but requires genuine effort. The choice of a reference that represents the state of the art (e.g., FAISS-GPU) ensures the target is meaningful.

B. Formal Definition

Let L_{curr} denote the current median latency of a system under benchmark B on hardware H. Let L_{ref} denote the latency of the reference state-of-the-art system under the same B on H. A ROTRTA assertion is:

$$\text{assert}\big(L_{\text{impl}} < L_{\text{ref}} - \epsilon\big) \tag{1}$$

where $\epsilon \geq 0$ is a margin chosen such that

$$\frac{L_{\text{curr}} - (L_{\text{ref}} - \epsilon)}{L_{\text{curr}}} \in [0.25,\ 0.50] \tag{2}$$

i.e., the assertion requires a 25–50% reduction relative to the current implementation. ϵ is typically set so that $L_{\text{ref}} - \epsilon$ is $1\,\mathrm{ms}$ or less below the measured reference, creating a margin that accounts for measurement noise while still requiring the implementation to genuinely match the reference class.

C. The Immutability Invariant

Once written, a ROTRTA assertion must never be modified to be less strict. This invariant is enforced by treating the test file as append-only after the initial commit. The justification is behavioral: if an agent (or developer) can weaken an assertion, it will do so whenever optimization becomes difficult. The immutability invariant forces the question back to the implementation.

D. Hardware Agnosticism

ROTRTA targets are expressed in absolute time units (milliseconds) rather than relative speedups, but must be calibrated to the target hardware class. For this work, calibration was performed on an NVIDIA RTX 4080 Laptop GPU running FAISS-GPU 1.13.2. The resulting thresholds are specific to that hardware class but remain reproducible within it.

E. Relationship to Test-Driven Development

ROTRTA shares TDD's test-first discipline and immutability of the test contract. It differs in three respects: (1) assertions encode *performance* rather than functional correctness; (2) the reference values are derived from external benchmarks rather than specifications; and (3) the optimization space is continuous and hardware-sensitive rather than discrete and portable. ROTRTA does not replace TDD; it complements it by adding a performance layer to the test suite.

F. Suitability for AI Agents

ROTRTA is particularly well-suited to AI agents for three reasons.

Binary oracle. An agent interacting with a build-and-test loop requires an unambiguous success signal. ROTRTA provides exactly this: the test suite either passes or fails. The agent need not form its own judgment about whether a speedup is "sufficient."

Composable improvements. A single ROTRTA target may require multiple coordinated changes. An agent can decompose the problem into candidate optimizations, apply them incrementally, and use the test oracle to validate each step. The binary signal guides search without requiring the agent to maintain a mental model of cumulative performance impact.

Prevention of premature satisficing. The immutability invariant means an agent cannot exit the optimization loop by rationalizing that "this is fast enough." The loop terminates only when the assertion passes.

IV. CatWhisper: System Design and Contrast with FAISS

CatWhisper is a C++ GPU vector similarity search library targeting production use. Its primary design axis — cross-vendor Vulkan compute rather than CUDA — leads to a set of architectural decisions that differ materially from FAISS, and which constitute both constraints and opportunities for optimization.

A. Vendor Neutrality via Vulkan

FAISS's GPU backend is built on CUDA and NVIDIA cuBLAS, limiting it to NVIDIA hardware. CatWhisper uses Vulkan 1.2 compute shaders compiled from GLSL to SPIR-V, and is tested on NVIDIA, AMD, and (via MoltenVK) Apple Silicon. Vulkan's lower-level API surface introduces more programmer responsibility — explicit synchronization, explicit memory type selection, explicit command buffer management — but eliminates the CUDA driver model's implicit stream management overhead in latency-sensitive paths.

B. Memory Layout: SoA vs. AoS

FAISS stores database vectors in Array-of-Structures (AoS) order:

$$\text{database}[\text{vector_idx} \times d + \text{dim_idx}] \tag{3}$$

Under AoS layout, when T threads in a warp each process a different vector, thread t reads dimension 0 of vector t, located at byte offsets $0, d \cdot s_{\text{fp16}}, 2d \cdot s_{\text{fp16}}, \ldots$ These are strided accesses: each warp-level read pulls T separate cache lines.

CatWhisper uses Structure-of-Arrays (SoA) order:

$$\text{database}[\text{dim_idx} \times C + \text{vector_idx}] \tag{4}$$

where $C \geq n$ is the allocated capacity (stride). When thread t reads dimension d of its assigned vector at position $\text{base} + t$, it accesses $\text{database}[d \times C + \text{base} + t]$. Consecutive threads differ by one vector index and thus by two bytes (fp16). All 32 threads in a warp read a contiguous 64-byte cache line — one read per dimension step, regardless of the number of threads. This is the maximum possible coalescing efficiency.

The SoA layout requires a transposition pass during `add()`, which CatWhisper implements as a dedicated compute shader (`transpose_add.comp`) dispatched on the GPU, adding no CPU cost.

C. Data Type: fp16 Storage, fp32 Accumulation

Both CatWhisper and FAISS support fp16 (half-precision) database vectors. CatWhisper stores vectors as `float16_t` in the SoA buffer and promotes each value to `float` before accumulation within the shader:

Listing 1. fp16-to-fp32 accumulation in GLSL

```
float diff = float(database[d*cap+pos])
           - float(query[d]);
sum += diff * diff;
```

This halves the database memory footprint, improving cache utilization, and cuts memory bandwidth by $2\times$ relative to fp32 storage — the dominant cost at large n.

D. Fused Distance and Top-K Shader

FAISS separates distance computation and top-k selection into distinct kernel invocations, requiring an intermediate distance buffer of size n and a global memory barrier between passes. CatWhisper fuses both operations into a single shader (`distance_topk_fused.comp`):

1) Each workgroup of 1,024 threads processes a tile of $C_t = 2{,}048$ vectors, computing distances and writing them into a shared-memory sort buffer (`sort_buf[2048]`).
2) An ascending bitonic sort over the 2,048-element shared buffer is performed in-place, with each thread executing two comparisons per stage.
3) The top-k entries (the first k elements of the sorted buffer) are written to a result buffer.

This fusion eliminates the intermediate distance buffer, reduces global memory traffic by $(n - k \cdot G)$ floats per query (where G is the number of workgroups), and avoids a global barrier. The CPU performs a final $O(kG \log k)$ max-heap merge over the $G \cdot k$ partial results.

E. Synchronization: Timeline Semaphores and Userspace Spin-Poll

FAISS-GPU uses CUDA stream synchronization (`cudaStreamSynchronize`), which issues a kernel and blocks the CPU until the GPU signals completion. On NVIDIA hardware, the CUDA driver maps a semaphore surface into the process address space, allowing `cudaStreamSynchronize` to poll in userspace without a syscall under favorable conditions.

Vulkan's traditional synchronization primitive, `VkFence`, requires `vkResetFences` before each reuse (a device-visible operation with ∼1–2 µs latency) and `vkWaitForFences` with a kernel-visible blocking wait (5–15 µs wake-up jitter on Linux).

CatWhisper replaces the fence with a Vulkan 1.2 timeline semaphore — a monotonically increasing 64-bit counter that signals after each `vkQueueSubmit` without requiring a reset — and polls it with `vkGetSemaphoreCounterValue` in a userspace busy-wait loop. On NVIDIA's proprietary driver, `vkGetSemaphoreCounterValue` reads a GPU-mapped host-visible surface that the GPU updates on completion, resulting in sub-microsecond detection latency with no kernel transitions. A spin limit (`SPIN_LIMIT = 2,000,000` iterations) prevents livelock: if the semaphore has not signaled within the limit, the code falls back to `vkWaitSemaphores` (blocking wait).

The combination eliminates two sources of latency present in the fence-based path: the reset operation and the kernel sleep/wake round-trip.

F. Zero-Copy Query and Result Paths

Query upload. A naive implementation allocates a staging buffer, copies the query vector into it, issues a `vkCmdCopyBuffer`, and submits and waits for the transfer — a full GPU round-trip per query. CatWhisper allocates the query buffer with `VMA_MEMORY_USAGE_GPU_TO_CPU` (HostCoherent) and maintains a persistent mapped pointer. Writing a query is a single `memcpy` from host memory to the mapped buffer, with no Vulkan call, no staging buffer, and no fence.

Result download. Distance and index result buffers are allocated as `VMA_MEMORY_USAGE_GPU_TO_CPU` (HostReadback, host-cached system RAM). After the GPU writes results, the CPU reads directly through the mapped pointer without a `vkCmdCopyBuffer` or fence wait. Host-cached system RAM is read efficiently by the CPU's prefetch hardware, comparable in bandwidth to reading from DRAM.

Query vector conversion. When fp16 mode is active, the query vector must be converted from `float32` (the public API type) to `float16` before the GPU shader can consume it. CatWhisper performs this conversion via SIMD intrinsics on the CPU during the `memcpy` into the mapped query buffer. The conversion path uses AVX-512 when available (`_mm512_cvtps_ph`, 16 elements per cycle), falls back to F16C (`_mm256_cvtps_ph`, 8 elements per cycle), and finally to a scalar implementation. On an AMD Ryzen 9 7945HX with AVX-512, converting a 256-element vector requires approximately 16 SIMD instructions.

G. Persistent Reusable Command Buffer

FAISS launches CUDA kernels directly; each launch queues work for the stream. Vulkan requires explicit command buffer recording. A naive implementation records a new command buffer for every search call: allocate, begin (with `ONE_TIME_SUBMIT_BIT`), bind pipeline, bind descriptor sets, push constants, dispatch, end, submit, wait, free. This recording overhead is 5 µs–15 µs on a modern system.

CatWhisper records the command buffer once with `flags=0` (reusable). On steady-state queries (same n, same k, no reallocation), the buffer is re-submitted without re-recording. The pipeline, descriptor sets, and push constants embedded in the buffer remain valid across queries as long as the backing `VkBuffer` handles do not change. A dirty flag tracks whether re-recording is needed (triggered by `add()` calls that cause reallocation).

H. Descriptor Set Caching

`vkUpdateDescriptorSets` binds buffer handles to shader binding points. Under the naive implementation, this is called once per search to ensure the descriptor set reflects the current buffer state. CatWhisper caches the buffer handles used in the last descriptor update and calls `vkUpdateDescriptorSets` only when a handle changes — which occurs at initialization and after a reallocation, but never on steady-state queries. This eliminates four driver round-trips per query in the common case.

I. Summary of Architectural Differences

Table I summarizes the key design differences between CatWhisper and FAISS.

TABLE I
CATWHISPER VS. FAISS: ARCHITECTURAL COMPARISON

Property	FAISS	CatWhisper
GPU API	CUDA (NVIDIA only)	Vulkan (cross-vendor)
DB memory layout	AoS (fp32/fp16)	SoA (fp16)
Distance+top-k	Separate passes	Fused shader
Sync primitive	CUDA fence	Timeline semaphore
CPU sync wait	Blocking syscall	Userspace spin-poll
Query upload	Staging + copy	Direct mapped write
Result readback	Device→host copy	Persistent mapped ptr
Cmd recording	Per-kernel launch	Persistent reusable buf
Descriptor update	Per-query	On buffer change only
fp16 conversion	cuBLAS/CUDA	AVX-512/F16C/scalar

V. APPLYING ROTRTA TO CATWHISPER

A. Development Timeline

CatWhisper's development proceeded in five discrete phases over approximately nine hours on 2026-02-23, visible in the git history:

1) **9c0311a — Initial commit** (19:11 UTC-6): Core Vulkan infrastructure, IndexFlat skeleton, AoS database, AoS shader with `local_size_x=256`. Performance: 10K×128 at 0.937 ms (18.0× slower than FAISS-GPU).
2) **4ec4585 — Fix GPU top-k sort** (19:47): Corrected ascending bitonic sort direction. First numerically correct GPU search.
3) **510144f — ROADMAP update** (20:06): Phase 1 marked functionally complete. 41/41 unit tests passing.
4) **2e3dbf6 — FAISS benchmark suite** (20:27): Added `bench_faiss.py` and measured reference numbers (FAISS-GPU 1.13.2 on RTX 4080 Laptop GPU). ROTRTA test file `tests/rotrta/rotrta_benchmark.cpp`

written and committed with assertions targeting the FAISS-GPU reference latency.

5) **f69cd36 — Phase 1 optimization sprint** (21:11): Human developer applied four coordinated optimizations (HostCoherent query buffer, persistent reusable command buffer, persistent fence, descriptor set caching). Result: 7.6× speedup at 10K×128 (0.937 ms→0.124 ms). ROTRTA status: 2/3 passing. The 10K×128 assertion (`median < 0.065 ms`) remained red at 0.072 ms–0.075 ms.
6) **bd85835 — ROTRTA agent session** (00:40 +1 day): Autonomous AI agent engaged with ROTRTA failure. All three assertions green.

B. The ROTRTA Assertions

The three ROTRTA assertions, as written in `tests/rotrta/rotrta_benchmark.cpp`, are:

Listing 2. ROTRTA assertions (immutable)

```
// BeatsFaissGpu_10Kx128
EXPECT_LT(median, 0.065)   // beat FAISS-GPU 0.065 ms

// BeatsFaissGpu_100Kx128
EXPECT_LT(median, 0.349)   // beat FAISS-GPU 0.350 ms

// BeatsFaissGpu_100Kx256
EXPECT_LT(median, 0.600)   // beat FAISS-GPU 0.601 ms
```

Each test runs 20 warmup queries followed by 100 timed single-query `search()` calls, records the median latency, and asserts it against the threshold. The warmup eliminates cold-start GPU scheduling jitter and pipeline cache misses.

C. Bottleneck Analysis: The Last-Mile Problem

After the Phase 1 human optimization sprint, the 10K×128 configuration presented a "last-mile" problem. The GPU kernel for 5 workgroups (each processing 1,024 vectors under the then-current CHUNK = 1,024 setting) executes in approximately 13 µs. Yet the measured end-to-end latency was 72 µs — meaning 59 µs was consumed by CPU overhead:

- `vkQueueSubmit`: 10 µs–30 µs (driver-side command packet serialization and queue ring write).
- `vkWaitForFences` (kernel blocking wait): 5 µs –15 µs (includes Linux scheduler wake-up latency).
- `vkResetFences`: 1 µs–2 µs.
- Scalar fp16 conversion: 2 µs–4 µs for 128 elements.
- Vector alloc for query buffer staging: 1 µs– 3 µs.

The ratio of GPU execution to CPU overhead was approximately 1:5. No single optimization could bridge the 7 µs gap to the 65 µs target; the ROTRTA oracle correctly forced a coordinated attack on all overhead sources simultaneously.

D. AI Agent Optimization Trace

The ROTRTA agent session proceeded as follows. The agent:

1) Read the test file, current measured latency (72 µs), and the failing assertion.
2) Read the shader source, pipeline submission code, and index implementation to profile the CPU path.
3) Identified the six-part optimization plan: SoA layout, CHUNK doubling, timeline semaphore, spin-poll, HostReadback buffers, and AVX-512 fp16 conversion.
4) Applied changes in a coordinated commit touching seven files: `shaders/distance_topk_fused.comp`, `shaders/distance_topk_batch.comp`, `src/gpu/pipeline.cpp`, `src/core/context_impl.hpp`, `src/indexes/index_flat.cpp`, `include/catwhisper/buffer.hpp`, and `src/gpu/buffer.cpp`.
5) Ran the ROTRTA suite. All three assertions green.

The agent's ability to identify that six independent improvements were needed simultaneously — none of which alone would have been sufficient — and to implement them coherently across seven files in a single session demonstrates the value of a binary pass/fail oracle. Without ROTRTA, any of the intermediate states (60 µs, 55 µs) might have been accepted as "sufficient."

E. Optimization Stack Detail

1) SoA Database Layout: The database buffer was restructured from AoS (`database[v * d + dim]`) to SoA (`database[dim * C + v]`). Two GPU shaders implement the maintenance operations: `transpose_add.comp` transposes incoming AoS vectors into the SoA buffer during `add()`, and `soa_repack.comp` changes the SoA stride when the database grows and buffers are reallocated. Both execute on the GPU asynchronously with respect to the CPU.

2) CHUNK Doubling: The shader tile size was increased from CHUNK = 1,024 to CHUNK = 2,048, and `local_size_x` from 512 to 1,024. This halved the number of workgroups dispatched for a given n: the 10K×128 case dropped from 10 to 5 workgroups, halving the number of partial results the CPU must merge. Each thread now computes distances for two vectors (positions base + tid and base + tid + 1,024), maintaining full occupancy.

3) Timeline Semaphore and Userspace Spin-Poll: `Context::Impl` was extended with a `VkSemaphore compute_timeline_semaphore` and a monotone counter `compute_timeline_value`. `submit_and_wait()` increments the counter, signals it via `VkTimelineSemaphoreSubmitInfo`, and polls with `vkGetSemaphoreCounterValue` in a tight loop (with `__builtin_ia32_pause()` for power efficiency). On NVIDIA's driver, this polls a host-visible GPU-mapped page with no syscall, yielding detection latency of approximately 1 µs–3 µs from GPU signal to CPU detection.

4) HostReadback Result Buffers: The output distance and index buffers were reallocated with `VMA_MEMORY_USAGE_GPU_TO_CPU` (host-cached system RAM). The GPU writes results to these buffers directly. The CPU reads through persistent mapped pointers without any `vkCmdCopyBuffer` or additional fence. Host-cached system RAM is efficient for CPU reads (via the CPU cache

hierarchy) and adequate for GPU writes on discrete GPUs (PCIE write-combining).

5) AVX-512 fp16 Conversion: The query conversion path was changed from scalar to SIMD:

Listing 3. AVX-512 fp16 conversion

```
#if defined(__AVX512F__)
for (; i + 16u <= n; i += 16u) {
    __m512 f32 = _mm512_loadu_ps(src + i);
    __m256i f16 = _mm512_cvtps_ph(f32,
                        _MM_FROUND_NO_EXC);
    _mm256_storeu_si256(
        (__m256i*)(dst + i), f16);
}
#endif
```

With `-march=native` (added to the CMake Release build flags), GCC targets AVX-512 on supported CPUs. Converting 128 fp32 values requires 8 AVX-512 iterations; converting 256 values requires 16.

VI. Experimental Results

A. Benchmark Configuration

Hardware. CPU: AMD Ryzen 9 7945HX3D (16 cores, AVX-512, F16C). GPU: NVIDIA GeForce RTX 4080 Laptop GPU (9,728 CUDA cores, 12 GB GDDR6, Vulkan 1.3, driver 580).

Software. CatWhisper compiled with GCC 12, `-O3 -march=native`, Vulkan SDK 1.3.280. FAISS 1.13.2, CUDA 12.0.

Methodology. 20 warmup queries followed by 100 timed queries. Metric: median single-query latency (milliseconds). Database: Gaussian random vectors (`std::normal_distribution`). $k = 10$.

B. Performance Across Optimization Phases

Table II shows median single-query latency at each development phase. Fig. **??** visualizes the progression.

TABLE II
Median Single-Query Latency (ms) by Phase

Phase	10K×128	100K×128	100K×256
Initial (9c0311a)	0.937	1.482	7.108
Phase 1 human (f69cd36)	0.124	0.619	2.002
ROTRTA agent (bd85835)	**0.053**	**0.056**	**0.106**
FAISS-CPU (1.13.2)	0.059	0.837	2.231
FAISS-GPU (1.13.2)	0.065	0.350	0.601
ROTRTA threshold	<0.065	<0.349	<0.600

C. Speedup Analysis

Table III decomposes the total speedup into contributions from each phase and benchmark.

CatWhisper after the ROTRTA session achieves latencies that are 1.2 ×–6.3 × lower than FAISS-GPU across the three benchmark configurations, despite operating over Vulkan rather than CUDA and on a cross-vendor codebase.

TABLE III
Cumulative Speedup vs. Initial Baseline

Phase	10K×128	100K×128	100K×256
Human Phase 1	7.6×	2.4×	3.6×
ROTRTA agent	2.3×	11.1×	18.9×
Total	**17.7**×	**26.5**×	**67.1**×
vs. FAISS-GPU	1.2× faster	6.3× faster	5.7× faster

D. Overhead Attribution

For the 10K×128 configuration, Table IV attributes the reduction from 72 µs (post-Phase 1) to 53 µs (post-ROTRTA) across the six optimizations.

TABLE IV
Estimated Latency Reduction per Optimization (10K×128)

Optimization	Estimated Saving
Timeline semaphore (no reset)	1 µs–2 µs
Userspace spin-poll (no sleep)	5 µs–15 µs
CHUNK 1024→2048 (fewer WGs)	1 µs–3 µs
HostReadback results (no copy)	1 µs–2 µs
Direct mapped query (no alloc)	1 µs–2 µs
AVX-512 fp16 convert	2 µs–3 µs
Total estimated	11 µs–27 µs
Observed reduction	19 µs

The observed reduction is within the estimated range. The dominant term is userspace spin-poll, which eliminates the Linux scheduler round-trip on the GPU completion signal path.

E. The 100K Anomaly

The 100K×128 and 100K×256 results show improvements of 11.1 × and 18.9 × respectively relative to the Phase 1 baseline — far exceeding the 2.3 × improvement at 10K×128. This is attributable to the SoA layout change, which produces much larger gains at higher n:

- At 10K×128, the entire database (10K × 128 × 2 B = 2.56 MB) fits in the GPU's L2 cache (4 MB on RTX 4080). The AoS-vs-SoA coalescing difference is partially masked by cache hits.
- At 100K×128 and 100K×256, the database (25.6 MB and 51.2 MB) exceeds L2 capacity. Every dimension step requires a global memory access. Under AoS, each warp-level access fetches 32 separate cache lines (one per thread, strided by $d \times 2$ B). Under SoA, each warp-level access fetches a single 64-byte cache line. The bandwidth improvement is approximately 32× in the worst case, manifesting as ∼10× observed speedup (limited by other latency terms).

This asymmetry would not have been predicted or discovered without ROTRTA forcing the optimizer to address all configurations simultaneously.

VII. Discussion

A. ROTRTA as a Forcing Function for Coordinated Optimization

The 10K×128 last-mile problem illustrated a property of ROTRTA: when no single optimization is sufficient, the binary pass/fail signal drives the agent to compose multiple improvements. A gradient-based approach (profiling plus incremental tuning) could converge on any single improvement and declare local optimality. ROTRTA cannot: the test is either green or red. This property is especially valuable when the optimizations are independent — they can be developed in parallel and tested together — but individually insufficient.

B. ROTRTA vs. Traditional Benchmarking

Traditional benchmarking measures performance after the fact and informs roadmap decisions. ROTRTA encodes the roadmap *as tests* and enforces it *at commit time*. This has two practical consequences:

1) Performance regressions are caught immediately (any commit that reduces performance below the threshold fails CI).
2) Targets are never silently abandoned. In a traditional workflow, a benchmark result that is "close enough" is often promoted to "good enough" without formal acknowledgment. ROTRTA makes this choice explicit: the threshold must be formally revised (a visible git commit) rather than quietly ignored.

C. Limitations

Hardware specificity. ROTRTA thresholds are calibrated to a specific hardware configuration. Assertions written for an RTX 4080 Laptop GPU will produce different pass/fail outcomes on an RTX 3070 or an AMD RX 7900 XTX. Hardware-portable ROTRTA requires either relative assertions (benchmarked against a co-located reference), runtime calibration, or hardware-specific threshold tables.

Measurement variance. High-frequency noise in GPU scheduling, thermal throttling, and OS scheduling can cause median latency to fluctuate by 10–20% across runs. The 100-sample median used here provides statistical stability, but a ROTRTA threshold set too aggressively relative to measurement variance risks non-deterministic pass/fail behavior. The 25–50% reduction guideline provides a practical safety margin.

Adversarial satisficing. An agent that can modify the test file could weaken a failing assertion to make it pass. The immutability invariant must be enforced by repository policy (branch protection, code review) rather than technical means.

Single-query focus. The assertions in this case study measure single-query latency. ROTRTA does not inherently enforce batch throughput, tail latency (P99), or memory consumption. Extending the methodology to multiple dimensions requires a corresponding increase in the number of assertions.

D. Applicability Beyond Vector Search

The ROTRTA pattern is domain-agnostic. It applies wherever:

- A state-of-the-art reference system exists and is measurable.
- Performance is primarily bottlenecked by implementation rather than algorithms.
- The implementation space admits multiple independent optimizations.
- An AI agent (or a human developer) will drive the optimization loop.

Candidate domains include database query engines, rendering pipelines, network data planes, and real-time signal processing.

E. Implications for AI-Assisted Engineering

The CatWhisper case demonstrates that an AI agent given a well-defined ROTRTA target can identify, plan, and execute a multi-file, multi-component optimization that human developers had approached but not completed. The key enabler was not the agent's superior knowledge of Vulkan internals (that knowledge was present in the Phase 1 human commits), but rather the agent's ability to enumerate the full optimization space, recognize that all six improvements were necessary, and execute them coherently across seven source files without losing global context. This is a task at which agents are strong (broad search) and humans are weak (cognitive load of holding many simultaneous constraints). ROTRTA makes the agent's task well-posed; without it, the agent would have no mechanism to determine when to stop.

VIII. Related Work

Vector similarity search. Johnson et al. [1] introduced FAISS. ScaNN [2] achieves high throughput via anisotropic quantization. Hnswlib [3] provides HNSW graph-based ANN search. DiskANN [4] targets billion-scale SSD-backed indexes. CatWhisper's contribution is cross-vendor Vulkan compute with a focus on single-query latency parity with CUDA-based systems.

GPU vector search. Faiss-GPU [1] uses CUDA and cuBLAS for batched search. GGNN [5] implements GPU-based HNSW. Lanns [6] provides GPU-accelerated SONG. None of these systems use Vulkan, and none report sub-70 µs single-query latency for 10K×128 under a single-query API.

GPU synchronization. The performance of GPU command submission and fence synchronization on modern drivers is analyzed in [9]. Timeline semaphore userspace polling is documented in the Vulkan specification [7] and confirmed by NVIDIA driver source analysis [8].

AI-assisted optimization. AlphaCode [10] and Codex [11] demonstrate LLM-driven code synthesis. SWE-bench [12] measures agent performance on software engineering tasks. Performance-oriented agent benchmarks are less well studied. ROTRTA proposes a concrete framework for AI-driven performance optimization that extends naturally to existing agent evaluation frameworks.

Test-driven development. Beck's TDD [13] is the antecedent methodology. Performance TDD has been explored in the context of database systems [14] and real-time systems, but without the SOTA-anchoring and immutability invariants that define ROTRTA.

IX. Conclusion

We have presented ROTRTA, a methodology for test-driven performance engineering in which latency assertions are anchored to state-of-the-art reference systems and are contractually immutable once written. We have demonstrated ROTRTA on CatWhisper, a cross-vendor Vulkan GPU vector similarity search library, achieving a $17.7\times$–$67.1\times$ total speedup over the initial implementation and causing CatWhisper to outperform FAISS-GPU — the industry standard — by $1.2\times$ to $6.3\times$ on three benchmark configurations.

The critical observation is that the final 7 µs gap to the ROTRTA threshold at 10K×128 required six simultaneous optimizations across seven source files, none of which was individually sufficient. The binary pass/fail ROTRTA oracle drove an AI agent to find and implement this compound solution where a human optimization sprint had stalled.

We believe ROTRTA fills a genuine gap in the software engineering toolbox: it makes performance targets as rigorous and enforceable as functional correctness requirements, and provides AI agents with the well-defined success criterion they need to perform genuine optimization rather than superficial improvement. As AI agents take on increasing roles in performance-critical software development, methodologies like ROTRTA that provide binary, reproducible, and meaningful success signals will become essential infrastructure.

CatWhisper is available at https://github.com/your-org/catwhisper under a dual MIT/Apache 2.0 license.

References

[1] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[2] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, and S. Kumar, "Accelerating large-scale inference with anisotropic vector quantization," in *Proc. ICML*, 2020.

[3] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2018.

[4] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Proc. NeurIPS*, 2019.

[5] J. Herrmann, I. Shieh, and T. Gross, "GGNN: Graph-based GPU nearest neighbor search," *IEEE Trans. Big Data*, 2021.

[6] T. Zhao, W. Zhang, and S. Wang, "SONG: Approximate nearest neighbor search on GPU," in *Proc. ICDE*, 2020.

[7] The Khronos Group, "Vulkan 1.3 Specification," 2024. [Online]. Available: https://registry.khronos.org/vulkan/specs/1.3/html/

[8] NVIDIA Corporation, "NVK Open-Source Vulkan Driver," 2024. [Online]. Available: https://gitlab.freedesktop.org/mesa/mesa

[9] F. Schenk, "Analyzing GPU command submission overhead in Vulkan," *GPU Pro 7*, CRC Press, 2020.

[10] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[12] C. E. Jiménez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv preprint arXiv:2310.06770*, 2023.

[13] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2003.

[14] S. Padmanabhan, T. Malkemus, A. Jhingran, and R. Agarwal, "Block oriented processing of relational database operations in modern computer architectures," in *Proc. ICDE*, 2001.